



NIO文件组件概述

北京理工大学计算机学院
金旭亮

概述



在Java IO中，主要使用File来封装文件与文件夹信息，到了NIO时代，则改用Path来完成相同的任务。

在NIO中，与路径相关的功能，被封装到了Paths这个工具类中。



为什么 NIO 要引入一个 Path 类来取代 File?

Files相关的类型，非常地“庞杂”.....

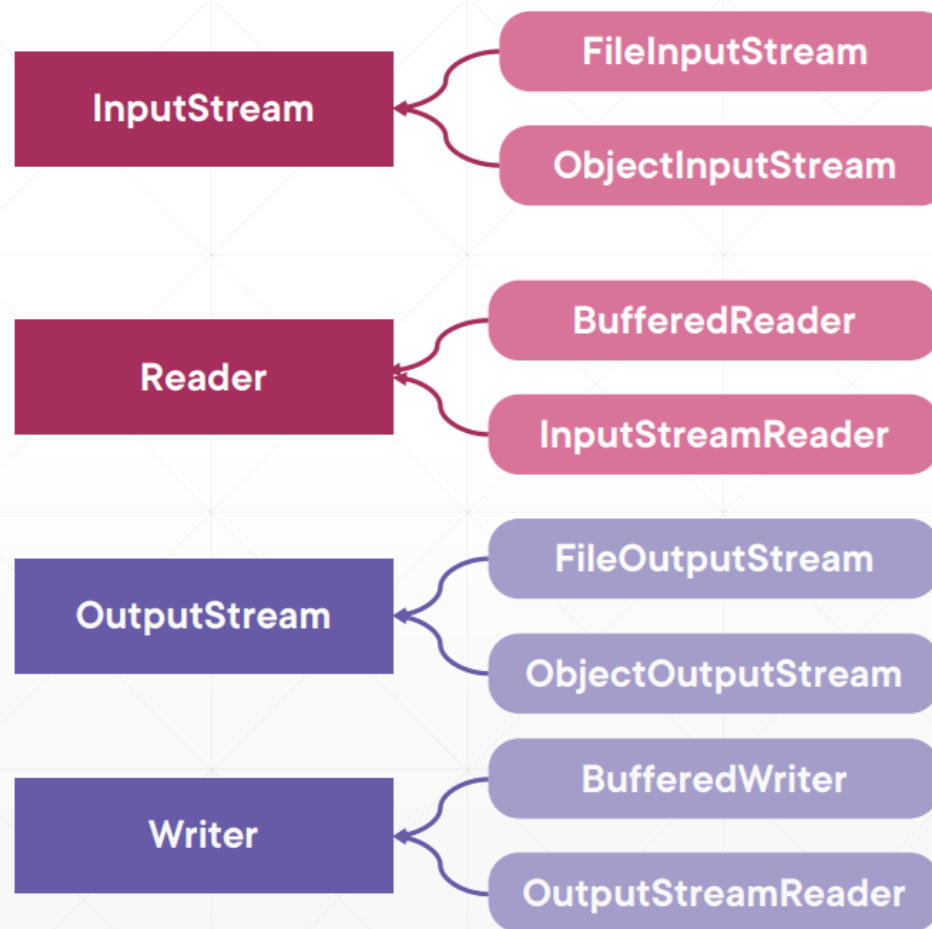
java.io

└─File.java

```
boolean delete()
```

```
boolean exists()
```

```
// etc.
```



File类的缺陷-1



File仅仅只是文件路径的一个封装，看上去“方法与属性”很多，但实际上却缺少一些重要的功能：



早期的File类，不抛出异常，当特定操作失败时，往往也不会给出详细的信息。



比如，`File.delete()`删除文件时，如果删除失败，它只返回一个false值，删除失败的具体原因，你是不知道的。



又如，`File.rename()`给文件改名时，在不同的平台上其表现并不一致。



还有，File不支持“symbolic link”特性，在Linux中，它是一种特殊的文件，用于引用另一个文件。

File类的缺陷-2



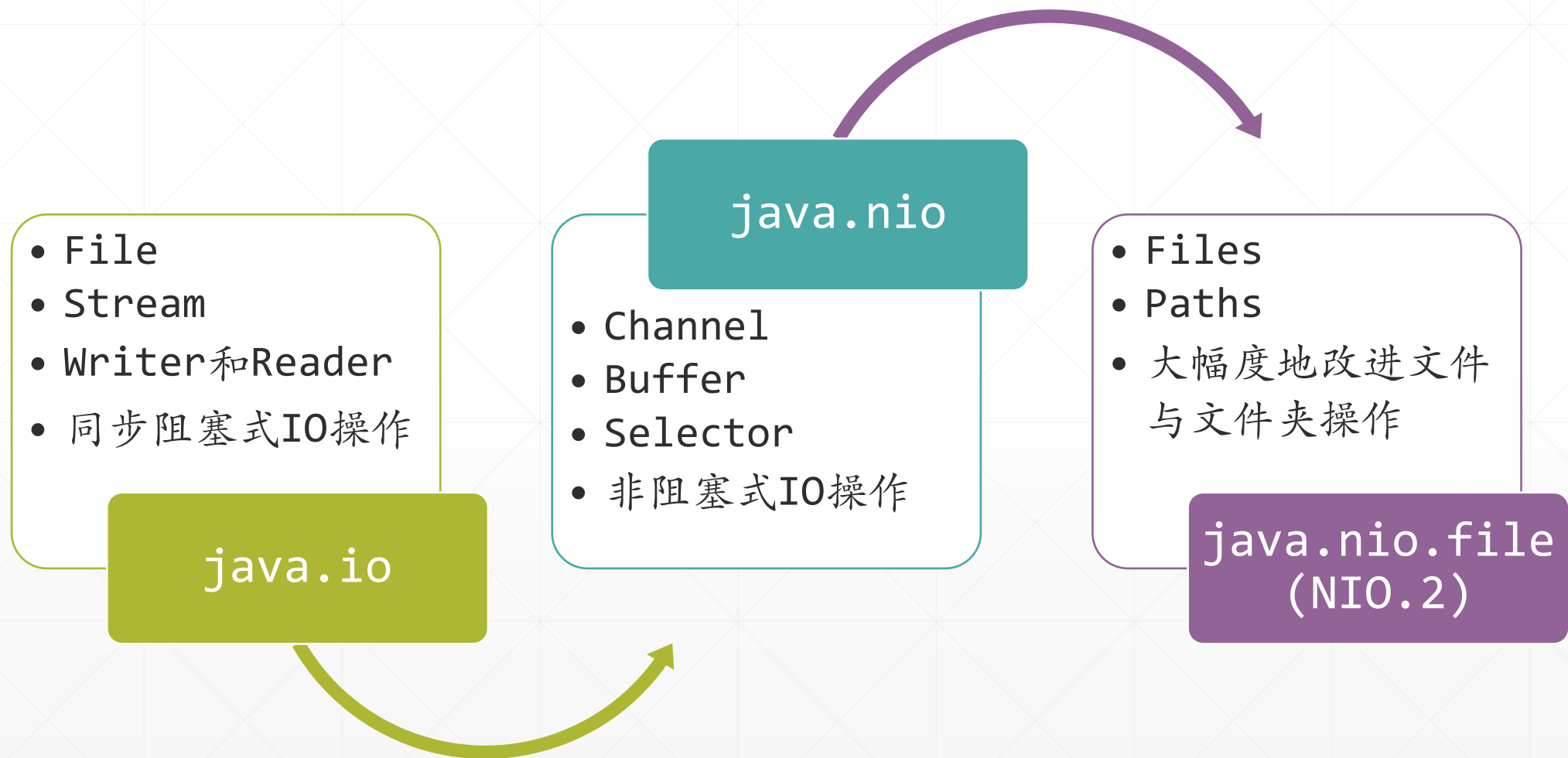
File类没有提供高效的方法提取文件的特定信息（比如文件的权限、所有者等等），每提取一个信息，都必须访问一次文件系统。



File类在处理大文件或包容很多文件和子文件夹的大文件夹时，性能不佳。

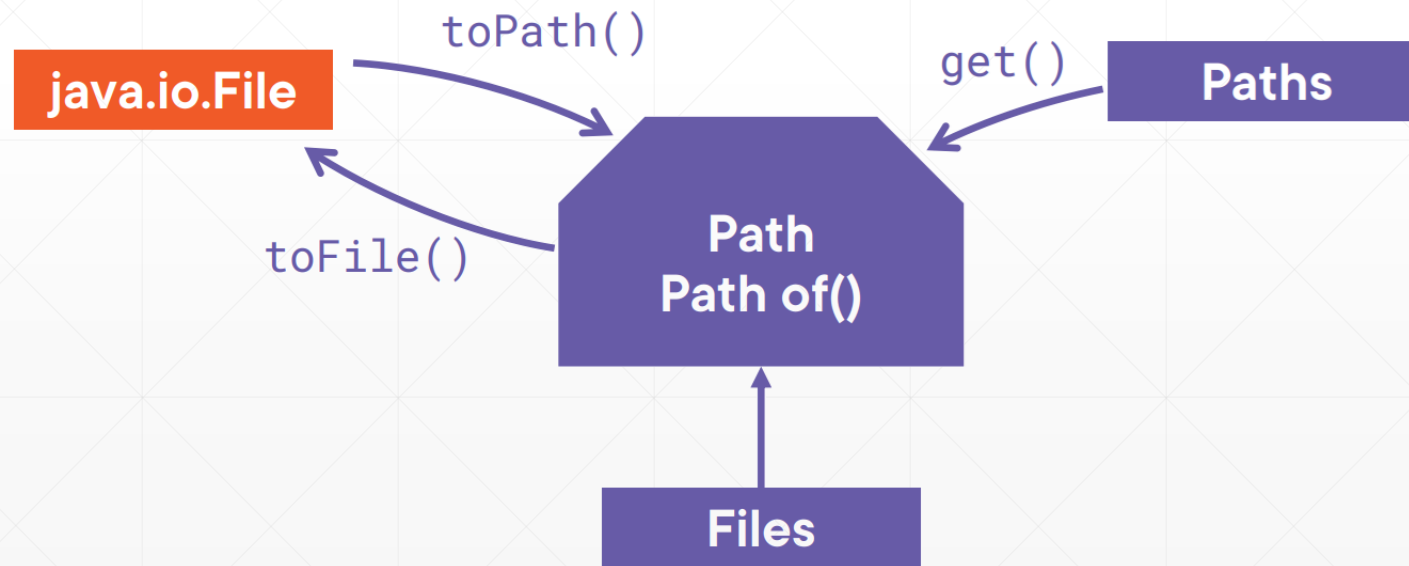
正是由于File存在着以上缺陷，所以NIO推出了Path来取代File。

Java IO组件的演化

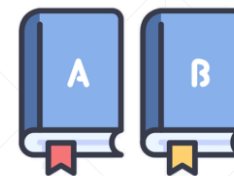




当前的Java应用，推荐优先选择NIO和NIO.2包中的组件，完成文件和文件系统的存取任务。



Files和Path是绝配!



```
copy(Path p, Path p1)  
delete(Path p)  
exists(Path p)  
readString(Path p)
```

Files